

Stream and lambdas

UA.DETI.POO

Lambda

Cálculo lambda

- ❖ As linguagens de programação funcional são baseadas no cálculo lambda (*cálculo- λ*).
 - Lisp, Haskell, Scheme
- ❖ O cálculo lambda pode ser visto como uma linguagem de programação abstrata em que funções podem ser combinadas para formar outras funções.

<https://www.baeldung.com/java-functional-programming>

Cálculo lambda

- ❖ **Ideia geral:** formalismo matemático
 - $x \rightarrow f(x)$ i.e. x é transformado em $f(x)$
- ❖ O cálculo lambda trata as funções como *elementos de primeira classe*:
 - podem ser utilizadas como argumentos e retornadas como valores de outras funções.
 - $x \rightarrow g(f(x))$ i.e., o resultado de $f(x)$ é transformado por $g()$

<https://www.baeldung.com/java-functional-programming>

<https://www.geeksforgeeks.org/java/lambda-expressions-java-8/>

Sintaxe

- ❖ Uma expressão lambda descreve uma função anónima. Representa-se na forma:

- (argument) -> (body)

- ❖ Pode ter zero, um, ou mais argumentos

- () -> { body }
 - (arg1, arg2...) -> { body }

- ❖ O tipo dos argumentos pode ser explicitamente declarado ou inferido

- (type1 arg1, type2 arg2...) -> { body }

- ❖ O corpo (body) pode ter uma ou mais instruções

```
(int a, int b) -> { return a + b; }
```

```
() -> System.out.println("Hello World");
```

```
(int a, int b) -> { return a + b; }
```

```
a -> return a*a    // um argumento – podemos omitir os parêntesis
```

```
(str) -> {  
    StringBuilder result = new StringBuilder();  
    for(int i = 0; i < str.length(); i++)  
        if(str.charAt(i) != ' ')  
            result.append( str.charAt(i) );  
    return result.toString();  
}
```

Exemplos

lambda expression	equivalent method
<code>() -> { System.gc(); }</code>	<code>void nn() { System.gc(); }</code>
<code>(int x) -> { return x+1; }</code>	<code>int nn(int x) return x+1; }</code>
<code>(int x, int y) -> { return x+y; }</code>	<code>int nn(int x, int y) { return x+y; }</code>
<code>(String... args) ->{return args.length;}</code>	<code>int nn(String... args) { return args.length; }</code>
<code>(String[] args) -> { if (args != null) return args.length; else return 0; }</code>	<code>int nn(String[] args) { if (args != null) return args.length; else return 0; }</code>

Lambda em Java e Python

1. Basic Lambda Expression

Python:

```
python

add = lambda x, y: x + y
print(add(2, 3)) # Output: 5
```

Java:

```
java

BinaryOperator<Integer> add = (x, y) -> x + y;
System.out.println(add.apply(2, 3)); // Output: 5
```

Referências a métodos

Referência a métodos

- ❖ São **um tipo especial de expressões lambda**.
 - Permitem substituir expressões lambda que invocam métodos existentes.

❖ Exemplos

`str -> System.out.println(str)`

`(s1, s2) -> {return s1.compareToIgnoreCase(s2); }`



`System.out::println`

`String::compareToIgnoreCase`

```
String[] names = { "Steve", "Rick", "Aditya", "Negan", "Lucy", "Sansa"};
Arrays.sort(names, String::compareToIgnoreCase);
for(String str: names){
    System.out.println(str);
}
```

Referências a métodos: 4 variedades

Kind	Syntax/Examples	Equivalent to
Reference to a static method	Class::staticMethod Math::abs Double::compare Math::random	(args) -> Class.staticMethod(args) (x) -> Math.abs(x) (x, y) -> Double.compare(x, y) () -> Math.random()
Reference to an instance method of a particular object	obj::method System.out::println "abcdef"::substring	(args) -> obj.method(args) (s) -> System.out.println(s) (a, b) -> "abcdef".substring(a, b)
Reference to an instance method of arbitrary object of a particular type	Type::method String::compareTo String::strip	(arg1, args) -> arg1.method(args) (s, t) -> s.compareTo(t) (s) -> s.strip()
Reference to a constructor	Class::new File::new int[]::new	(args) -> new Class(args) (name) -> new File(name) (size) -> new int[size]

[Method references \(Java tutorial\)](#)

Utilização de expressões lambda

❖ Iterar sobre Java Collections

```
// solução 1
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
for (Integer n: list) {
    System.out.println(n);
}
```

```
// solução 2
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
list.forEach(n -> System.out.println(n));
```

```
// solução 3, method reference (:: double colon operator)
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
list.forEach(System.out::println);
```

Como usar?

- ❖ Uma expressão lambda não pode ser declarada isoladamente

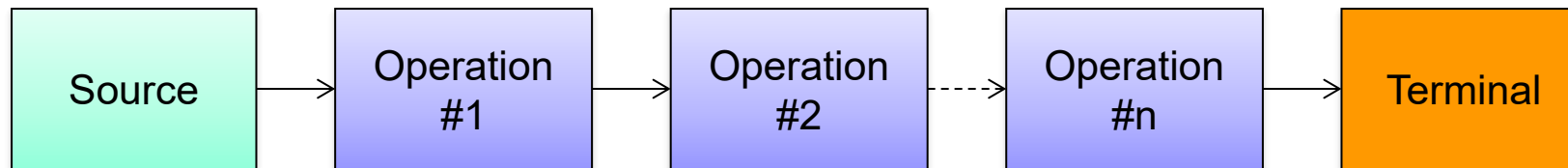
`(n) -> (n % 2) == 0; // Erro de compilação`

- ❖ Precisamos de outro mecanismo adicional  **Detalhes mais frente**
 - Interfaces funcionais
 - onde as expressões lambda passam a ser implementações de métodos abstratos.
 - O compilador Java converte uma expressão lambda num método da classe (isto é um processo interno).

Streams

Stream API

- ❖ introduced in [Java 8](#),
- ❖ **Stream API** is used to process collections of objects.
 - **process collections of data in a functional style**. Instead of writing loops, you describe **what operations you want to perform** on the data.
- ❖ **stream** as a pipeline where data flows through multiple operations like **filtering, mapping, sorting, and collecting**.



Relies on chaining: **Source.Op1.Op2 .. .Terminal**

<https://www.baeldung.com/java-8-streams>

<https://www.geeksforgeeks.org/stream-in-java/>

Traditional vs stream

```
List<Integer> numbers =  
Arrays.asList(1,2,3,4,5,6);  
  
List<Integer> even = new ArrayList<>();  
for(int n : numbers){  
    if(n % 2 == 0){  
        even.add(n);  
    }  
}
```

```
List<Integer> even = numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .toList();
```

Foreach: Iterar sobre coleções

Iterator

```
List<String> names = Arrays.asList("Ana", "Ze", "Rui");  
Iterator<String> it = names.iterator();  
while (it.hasNext())  
    System.out.println(it.next());
```

ciclo "for each"

```
List<String> names = Arrays.asList("Ana", "Ze", "Rui");  
for (String name : names)  
    System.out.println(name);
```

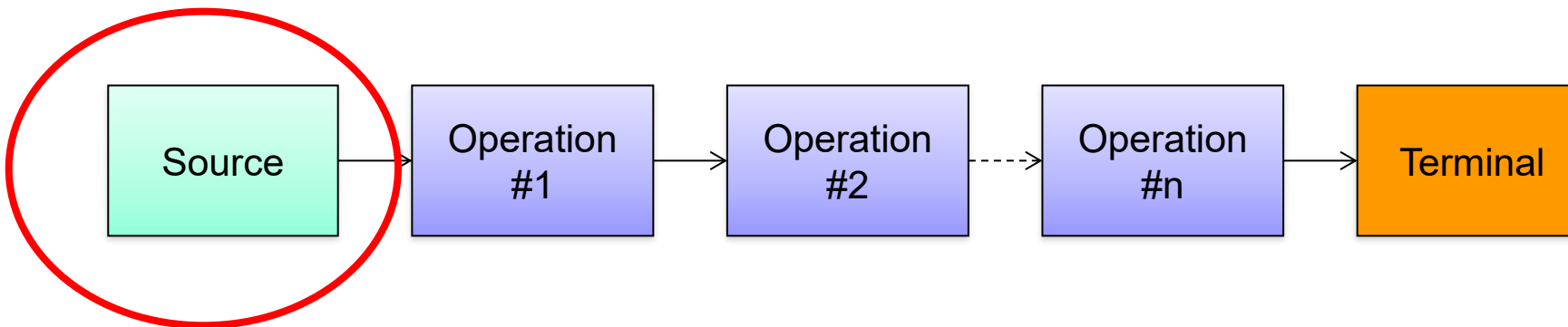
Stream operations: *Aggregate operations*

```
List<String> names = Arrays.asList("Ana", "Ze", "Rui");  
names.forEach(s -> System.out.println(s)); // forEach com lambda  
names.forEach(System.out::println); // forEach com referência de método
```

Sources of Streams: java.util.stream

❖ Streams sources include:

- From a `Collection` via the `stream()` and `parallelStream()` methods;
- From an `Array` via `Arrays.stream(Object[])`;
- *and many more (files, random, ..)*



Sources of Streams: java.util.stream

```
List<Person> persons = bag.getItems().stream()  
    .filter(item -> item instanceof Person)  
    .map(item -> (Person) item)  
    .toList();
```

```
for (Person item : persons) {  
    System.out.println(item);  
}
```

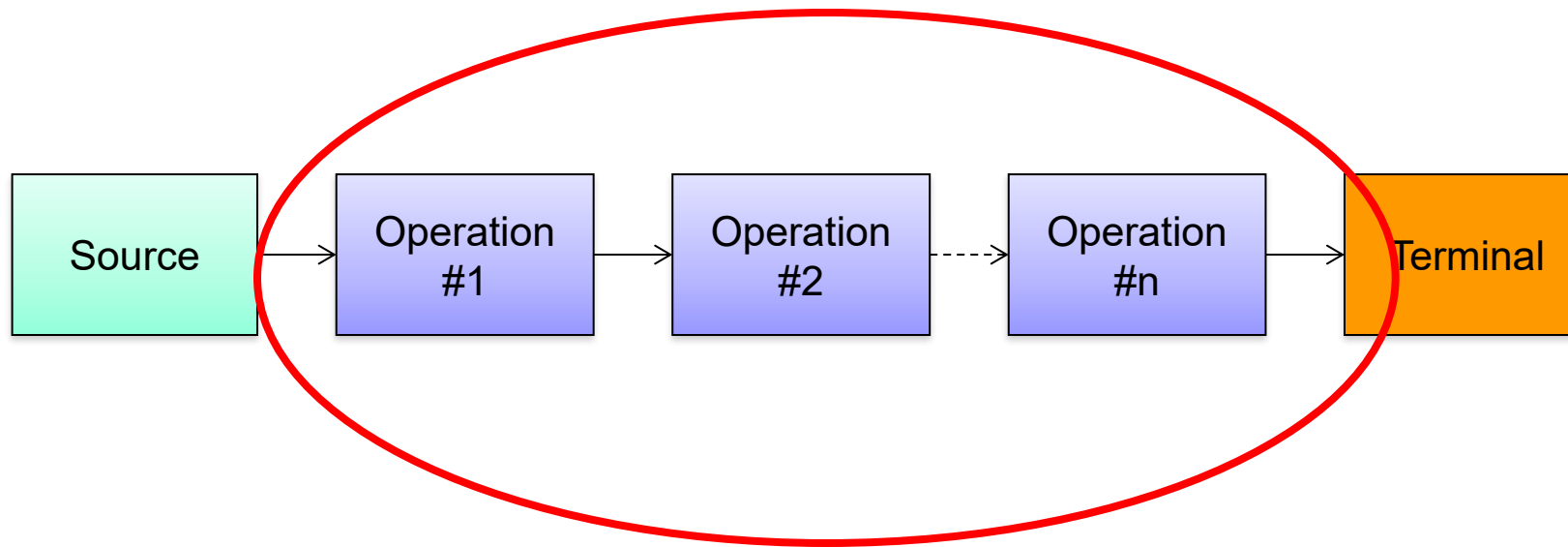
Sources of Streams: java.util.stream

```
Stream<String> streamBuilder =  
Stream.<String>builder().add("a").add("b").add("c").build();
```

```
Path path = Paths.get("C:\\file.txt"); Stream<String> streamOfStrings =  
Files.lines(path); Stream<String> streamWithCharset = Files.lines(path,  
Charset.forName("UTF-8"));
```

```
Stream<String> streamGenerated = Stream.generate(() -> "element").limit(10);
```

Handling streams (i.e. do something)



java.util.stream – Intermediate operations

- ❖ Methods are chained together.
- ❖ Intermediate operations transform a stream into another stream.
- ❖ It enables the concept of filtering where one method filters data and passes it to another method after processing.

java.util.stream – Intermediate operations

❖ filter

- excludes all elements that don't match a Predicate

❖ map

- perform transformation of elements using a Function

❖ flatMap

- applies a one-to-many transformation to the elements of the stream, and then flattens the resulting elements into a new Stream

❖ peek

- performs some action on each element

❖ distinct

- excludes all duplicate elements (equals())

❖ sorted

- ordered elements (Comparator)

❖ Limit

- maximum number of elements

❖ skip

- discard first n elements

❖ (and many more -> see `java.util.stream.Stream<T>`)

java.util.stream – Intermediate operations

```
List<Person> people = ...;  
Stream<Person> tenPersonsOver18 = people.stream()  
    .filter(p -> p.getAge() > 18)  
    .limit(10);
```

Process the stream

```
List<Person> persons = bag.getItems().stream()
    .filter(item -> item instanceof Person)
    .map(item -> (Person) item)
    .toList();

for (Person item : persons) {
    System.out.println(item);
}
```

Java Stream Filter with Lambda Expression

```
public class Customer {  
    private String name;  
    private int points;  
    //Constructor and standard getters  
  
    public boolean hasOverHundredPoints() {  
        return this.points > 100;  
    }  
}
```

```
List<Customer> customersWithMoreThan100Points =  
customers  
    .stream()  
    .filter(c -> c.getPoints() > 100)  
    .collect(Collectors.toList());
```

```
List<Customer> customersWithMoreThan100Points =  
customers  
    .stream()  
    .filter(Customer::hasOverHundredPoints)  
    .collect(Collectors.toList());
```

```
List<Customer> charlesWithMoreThan100Points = customers  
    .stream()  
    .filter(c -> c.getPoints() > 100 && c.getName().startsWith("Charles"))  
    .collect(Collectors.toList());
```

```
assertThat(charlesWithMoreThan100Points).hasSize(1);  
assertThat(charlesWithMoreThan100Points).contains(charles);
```

<https://www.baeldung.com/java-stream-filter-lambda>

Java Stream Filter with Lambda Expression

```
public class Customer {  
    private String name;  
    private int points;  
    //Constructor and standard getters  
  
    public boolean hasOverHundredPoints() {  
        return this.points > 100;  
    }  
}
```

```
List<Customer> customersWithMoreThan100Points =  
customers  
    .stream()  
    .filter(c -> c.getPoints() > 100)  
    .collect(Collectors.toList());
```

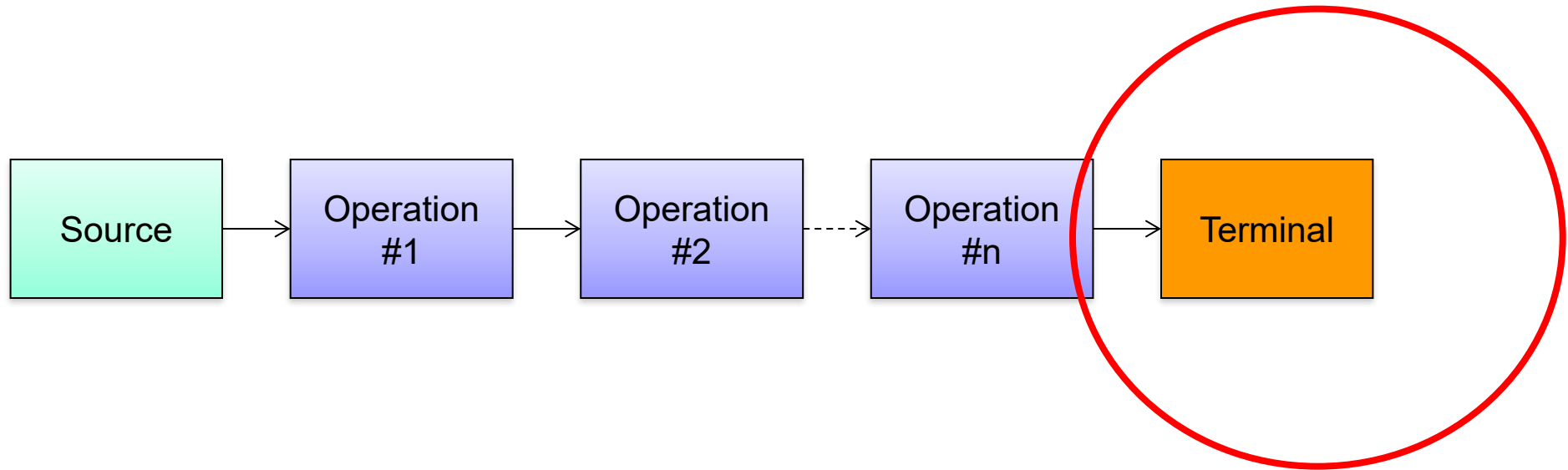
```
List<Customer> customersWithMoreThan100Points =  
customers  
    .stream()  
    .filter(Customer::hasOverHundredPoints)  
    .collect(Collectors.toList());
```

```
List<Customer> charlesWithMoreThan100Points = customers  
    .stream()  
    .filter(c -> c.getPoints() > 100 && c.getName().startsWith("Charles"))  
    .collect(Collectors.toList());
```

```
assertThat(charlesWithMoreThan100Points).hasSize(1);  
assertThat(charlesWithMoreThan100Points).contains(charles);
```

<https://www.baeldung.com/java-stream-filter-lambda>

End the processing



java.util.stream – Terminating operations

❖ Reducers

- `reduce()` (*- allows immutable objects), `count()`, `findAny()`, `findFirst()`

❖ Collectors

- `collect()` (*- only works with mutable objects)

❖ `forEach`

❖ iterators

```
// Accumulate names into a List
List<Person> people = ...;
List<String> names =
    people.stream()
        .map(Person::getName)
        .collect(Collectors.toList())
    ;
```

Get the results: in the example a new collection

```
List<Person> persons = bag.getItems().stream()
    .filter(item -> item instanceof Person)
    .map(item -> (Person) item)
    .toList();

for (Person item : persons) {
    System.out.println(item);
}
```

Some “curiosities”

❖ Can run **parallel streams**

```
nums.parallelStream()  
    .forEach(System.out::println);
```

Streams vs collections

- ❖ Collection → stores data
- ❖ A **collection** is a data structure used to **store and manage objects**.
- ❖ collections have a finite size (bounded)
- ❖ Stream → processes data
- ❖ A **stream** is a **sequence of elements used for computation**, not storage.
- ❖ Possibly **unbounded** streams 0zneed not a finit size
- ❖ **Functional in nature**
- ❖ An operation on a stream produces a result **but does not modify its source**.
- ❖ **Intermediate operations are always lazy.**
- ❖ elements **only visited once during the life of a stream**.
 - New visit implies new stream

Streams vs collections

Feature	Collections	Streams
Purpose	Store data	Process data
Nature	Data structure	Computation pipeline
Modifiable	Yes	No
Traversal	Multiple times	Usually once
Execution	Immediate	Lazy (most operations)
Iteration	External iteration (loops)	Internal iteration

Process, not store

```
List<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");
```

Name exists

```
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Stream doesn't

Only one

```
numbers.size();  
numbers.contains(3);  
numbers.get(0);
```

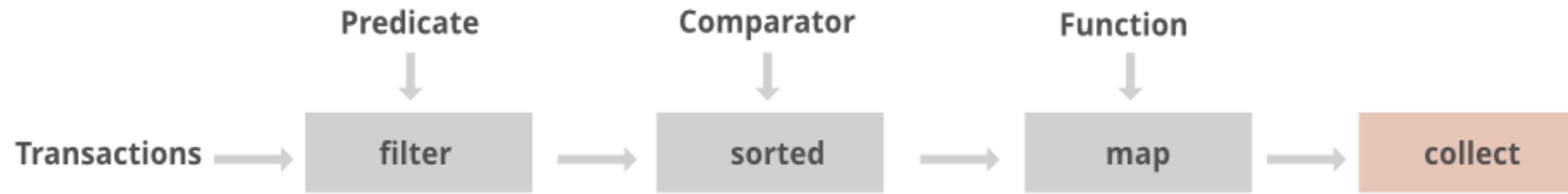
Numbers still exists

```
Stream<Integer> s = numbers.stream();  
s.forEach(System.out::println);
```

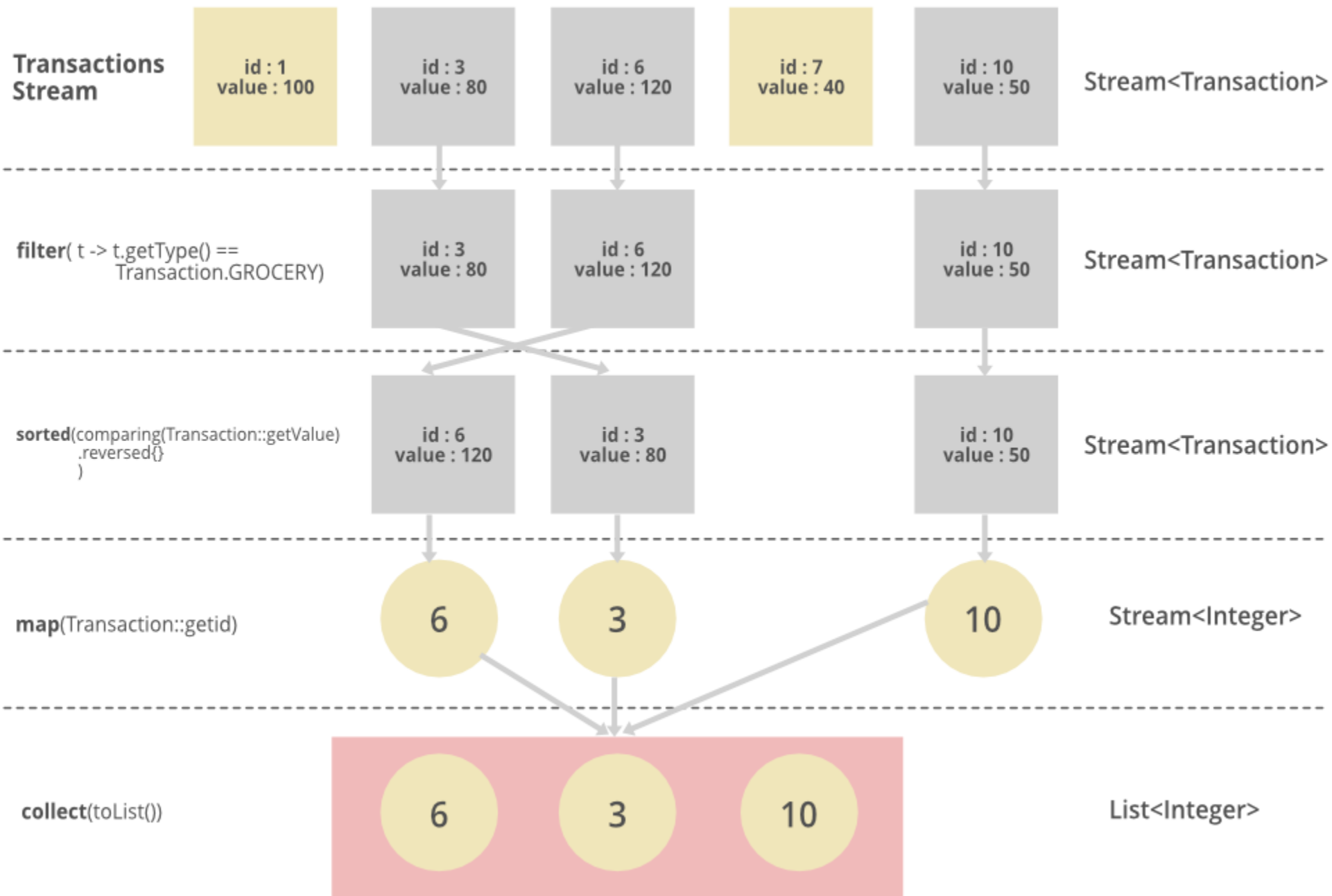
```
// This will throw an exception  
s.forEach(System.out::println);
```

A terminal stream “disappears”

Java Stream: Real-life Examples



```
List<Integer> transactionsIds =  
transactions.stream()  
  .filter(t -> t.getType() == Transaction.GROCERY)  
  .sorted(comparing(Transaction::getValue).reversed())  
  .map(Transaction::getId)  
  .collect(toList());
```



Common stream operations

- ❖ filter
- ❖ map
- ❖ sorted
- ❖ groupingBy
- ❖ collect **–or–** reduce

- ❖ cover **80–90% of real usage.**

filter(): Select elements that match a condition

- ❖ Filtering a stream of data is the first natural operation that we would need.
- ❖ Stream interface exposes a filter method that takes in a Predicate that allows us to use lambda expression to define the filtering criteria:

```
List<Integer> nums = List.of(10, 15, 20, 25);
```

```
nums.stream()  
    .filter(n -> n > 15)  
    .forEach(System.out::println);
```

```
20
```

```
25
```

map(): Transform each element

- ❖ allows us to apply a function that takes in a parameter of one type and returns something else.

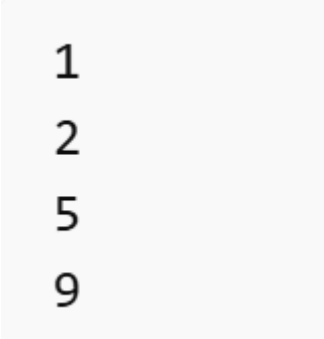
```
List<String> names = List.of("john",  
"mary", "alex");
```

```
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

```
JOHN  
MARY  
ALEX
```

sorted(): Sort elements

```
List<Integer> nums =  
List.of(5,2,9,1);  
  
nums.stream()  
    .sorted()  
    .forEach(System.out::println);
```



1
2
5
9

collect(): Convert the stream back to a collection

- ❖ The Stream API provides several “terminal” operations.
- ❖ The collect() method is one of those, which allows us to collect the results of the operations:

```
List<Integer> nums =  
List.of(1,2,3,4,5);
```

```
List<Integer> result = nums.stream()  
    .filter(n -> n % 2 == 1)  
    .toList();
```

```
[1,3,5]
```

Working with Objects

```
class Person {  
    String name;  
    int age;  
  
    Person(String name, int age){  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
List<Person> people = List.of(  
    new Person("Alice", 25),  
    new Person("Bob", 17),  
    new Person("Charlie", 30)  
);  
  
people.stream()  
    .filter(p -> p.age >= 18)  
    .map(p -> p.name)  
    .forEach(System.out::println);
```

```
Alice  
Charlie
```

Counting Elements

```
long count = nums.stream()  
    .filter(n -> n > 3)  
    .count();
```

```
System.out.println(count);
```

2

Finding Elements

```
Optional<Integer> first =  
    nums.stream()  
        .filter(n -> n > 3)  
        .findFirst();  
  
System.out.println(first.get());
```

Examples

Finding First Match

```
Optional<Integer> result =  
    numbers.stream()  
        .filter(n -> n > 10)  
        .findFirst();  
  
result.ifPresent(System.out::println);
```

Collecting into Map

```
Map<String, Integer> map =  
    people.stream()  
        .collect(Collectors.toMap(  
            p -> p.name,  
            p -> p.age  
        ));
```

```
System.out.println(map);
```

```
{Alice=25, Bob=30, Charlie=20}
```

Grouping Data

```
Map<Integer, List<Person>> grouped =  
    people.stream()  
        .collect(Collectors.groupingBy(p ->  
p.age));
```

```
System.out.println(grouped);
```

```
{  
  25=[Person(Alice)],  
  30=[Person(Bob)],  
  20=[Person(Charlie)]  
}
```

Remove Duplicates

```
List<Integer> numbers =  
List.of(1,2,2,3,3,4);
```

```
List<Integer> unique = numbers.stream()  
    .distinct()  
    .toList();
```

```
System.out.println(unique);
```

```
[1, 2, 3, 4]
```

Flatten Nested Collections (flatMap)

```
List<List<Integer>> list = List.of(  
    List.of(1,2),  
    List.of(3,4),  
    List.of(5,6)  
);  
  
List<Integer> result = list.stream()  
    .flatMap(List::stream)  
    .toList();  
  
System.out.println(result);
```

```
[1, 2, 3, 4, 5, 6]
```

Finding Maximum / Minimum Object

```
Optional<Person> oldest =  
    people.stream()  
        .max(Comparator.comparing(p -> p.age));  
  
oldest.ifPresent(p -> System.out.println(p.name));
```

Summing Values

```
int totalAge =  
    people.stream()  
        .collect(Collectors.summingInt(p -> p.age));  
  
System.out.println(totalAge);
```

Mapping Inside Grouping

```
Map<String, List<String>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            e -> e.department,  
            Collectors.mapping(e -> e.name, Collectors.toList())  
        ));
```

```
IT -> [Alice, Bob]  
HR -> [John]
```

Check conditions

```
boolean allAdults =  
    people.stream()  
        .allMatch(p -> p.age >= 18);
```

```
boolean noneTeen =  
    people.stream()  
        .noneMatch(p -> p.age < 13);
```

```
boolean hasMinor =  
    people.stream()  
        .anyMatch(p -> p.age < 18);
```

Does what?

```
List<String> result =  
    people.stream()  
        .filter(p -> p.age >= 18)  
        .sorted(Comparator.comparing(p -> p.age))  
        .map(p -> p.name)  
        .toList();
```

```

public class StreamIntermediateOperationsExample {
    public static void main(String[] args) {
        // List of lists of names
        List<List<String>> listOfLists = Arrays.asList(
            Arrays.asList("Reflection", "Collection", "Stream"),
            Arrays.asList("Structure", "State", "Flow"),
            Arrays.asList("Sorting", "Mapping", "Reduction", "Stream")
        );

        // Create a set to hold intermediate results
        Set<String> intermediateResults = new HashSet<>();

        // Stream pipeline demonstrating various intermediate operations
        List<String> result = listOfLists.stream()
            .flatMap(List::stream)           // Flatten the list of lists into a single stream
            .filter(s -> s.startsWith("S")) // Filter elements starting with "S"
            .map(String::toUpperCase)        // Transform each element to uppercase
            .distinct()                      // Remove duplicate elements
            .sorted()                        // Sort elements
            .peek(s -> intermediateResults.add(s)) // Perform an action (add to set) on each element
            .collect(Collectors.toList());   // Collect the final result into a list

        System.out.println("Intermediate Results:"); // Print the intermediate results
        intermediateResults.forEach(System.out::println);

        System.out.println("Final Result:"); // Print the final result
        result.forEach(System.out::println);
    }
}

```

Output

Intermediate Results:

STRUCTURE

STREAM

STATE

SORTING

Final Result:

SORTING

STATE

STREAM

STRUCTURE

```

public class StreamIntermediateOperationsExample {
    public static void main(String[] args) {
        // List of lists of names
        List<List<String>> listOfLists = Arrays.asList(
            Arrays.asList("Reflection", "Collection", "Stream"),
            Arrays.asList("Structure", "State", "Flow"),
            Arrays.asList("Sorting", "Mapping", "Reduction", "Stream")
        );

        // Create a set to hold intermediate results
        Set<String> intermediateResults = new HashSet<>();

        // Stream pipeline demonstrating various intermediate operations
        List<String> result = listOfLists.stream()
            .flatMap(List::stream)           // Flatten the list of lists into a single stream
            .filter(s -> s.startsWith("S")) // Filter elements starting with "S"
            .map(String::toUpperCase)        // Transform each element to uppercase
            .distinct()                      // Remove duplicate elements
            .sorted()                        // Sort elements
            .peek(s -> intermediateResults.add(s)) // Perform an action (add to set) on each element
            .collect(Collectors.toList());   // Collect the final result into a list

        System.out.println("Intermediate Results:"); // Print the intermediate results
        intermediateResults.forEach(System.out::println);

        System.out.println("Final Result:"); // Print the final result
        result.forEach(System.out::println);
    }
}

```

Some examples using a list of strings

```
public static void listExample() {
    List<String> words = new ArrayList<String>();
    words.add("Prego");
    words.add("no");
    words.add("Prato");

    // Java 8 has a forEach method as part of the Iterable<T> interface
    words.stream().forEach(n -> System.out.print(n + " "));
    System.out.println();

    // but in Java 8, why use a lambda when you can refer directly to the appropriate function?
    words.stream().forEach(System.out::print);
    System.out.println();

    // Let's introduce a call on map to transform the data before it is printed
    words.stream().map(n -> n + " ").forEach(System.out::print);
    System.out.println();

    // obviously these chains of calls can get long, so the convention is
    // to split them across lines after the call on "stream":
    words.stream()
        .map(n -> n + " ")
        .forEach(System.out::print);
    System.out.println();
}
```

```
Prego no Prato
Prego no Prato
Prego no Prato
PregonoPrato
Prego no Prato
Prego no Prato
```

Some examples with an array of int

```
public static void arraysExample() {
    int[] numbers = {3, -4, 8, 73, 507, 8, 14, 9, 3, 15, -7, 9, 3, -7, 15};

    // want to know the sum of the numbers? It's now built in
    int sum = Arrays.stream(numbers)
        .sum();
    System.out.println("sum = " + sum);

    // how about the sum of the even numbers?
    int sum2 = Arrays.stream(numbers)
        .filter(i -> i % 2 == 0)
        .sum();
    System.out.println("sum of evens = " + sum2);

    // how about the sum of the absolute value of the even numbers?
    int sum3 = Arrays.stream(numbers)
        .map(Math::abs)
        .filter(i -> i % 2 == 0)
        .sum();
    System.out.println("sum of absolute value of evens = " + sum3);

    // how about the same thing with no duplicates?
    int sum4 = Arrays.stream(numbers)
        .distinct()
        .map(Math::abs)
        .filter(i -> i % 2 == 0)
        .sum();
    System.out.println("sum of absolute value of distinct evens = " + sum4);
}
```

```
sum = 649
sum of evens = 26
sum of absolute value of evens = 34
sum of absolute value of distinct evens = 26
```

```

class GFG {
    // Method to filter strings of a given length and convert them to uppercase
    private static List<String> filterAndConvertToUpper(Stream<String> stream, int length)    {
        return stream.filter(s -> s.length() == length)
            .map(String::toUpperCase)
            .collect(Collectors.toList());
    }

    public static void main(String[] args)    {

        // Replace with the actual file path
        String fileName = "path/to/your/file.txt";

        // Step 1: Create a Stream of lines from the file
        try (Stream<String> lines
            = Files.lines(Paths.get(fileName))) {

            List<String> filteredStrings
                = filterAndConvertToUpper(lines, 5);
            System.out.println(
                "Filtered strings with length 5 (converted to uppercase): "
                + filteredStrings);
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

File Read Operation

```
// Driver Class
class GFG {
    // main function
    public static void main(String[] args) {
        String[] words = { "Geeks", "for", "Geeks", "Hello", "World" };

        // Replace with the
        // actual file path

        String fileName = "path/to/your/file.txt";

        // Step 1: Create a PrintWriter to write to the
        // file
        try (PrintWriter pw
            = new PrintWriter(Files.newBufferedWriter(
                Paths.get(fileName)))) {

            // Step 2: Use Stream to write each word to the file
            Stream.of(words).forEach(pw::println);

            // Step 3: Print success message to the console
            System.out.println(
                "Words written to the file successfully.");
        }
        catch (IOException e) {
            // Step 4: Handle any IO exception that occurs
            // during the file writing process
            e.printStackTrace();
        }
    }
}
```

File Write Operation

Sumário

- ❖ Lambda expressions
- ❖ JAVA Stream API